基于紫光同创 FPGA 的 RV32I 五级流水 RISC-V 处理器与边缘 AI 加速单元

—— 2025 紫光同创杯 FPGA 创新大赛 · 赛题一 设计报告

项目代号	PGT-RV32I-AI-01
设计语言	Verilog-2001（可综合）
目标器件	紫光同创 PGL22G 系列（板卡待定，工程参数化）
报告版本	V1.0 2025-11

摘　要

本报告介绍一套面向紫光同创 FPGA 的 RISC-V 处理器与边缘 AI 加速单元。处理器采用五级流水（IF/ID/EX/MEM/WB），实现 RV32I 基础整数指令集、Zicsr 控制状态寄存器与 M 扩展乘除法，配套两路组相联的指令/数据 Cache、BTB 分支预测以及完整的机器模式异常与中断机制；SoC 集成 UART、GPIO、64 位定时器与 memory-mapped 的 INT8 矩阵-向量加速器。

设计以“可综合、可验证、可移植”为目标：指令/数据主存为 128 位宽的 BRAM 风格存储器，Cache 一次读回整行（16 B），缺失代价 3 拍；全部 RTL 通过 Icarus Verilog 与 Vivado xsim 两套仿真器交叉验证，15 项自检测试全部通过。CoreMark 2K performance run 的 CRC 与官方值一致，计时区 357 397 拍，折算约 2.80 CoreMark/MHz；MNIST 手写数字识别在加速器上实测 19/20（95%），与 Python 端 INT8 仿真 96.19% 相当。

综合参考结果：全 SoC 约 10 919 LUT、3 726 FF、28 个 BRAM、20 个 DSP，在 Artix-7 上 10 ns 约束下 WNS 为 −3.10 ns（约 76 MHz，未布局布线的估算值）。报告最后给出板卡适配、卷积/AI 算子扩展与频率优化的后续路线。

关键词：RISC-V；五级流水；Cache；分支预测；FPGA；边缘 AI 加速；CoreMark

1 赛题要求与完成情况

1.1 赛题任务

赛题要求使用 Verilog/SystemVerilog，基于紫光同创 FPGA 完成一套 RISC-V 处理器的设计，包含三项基础任务与三项高阶任务：

· 基础任务：① RV32I 通用处理单元；② 不少于三级的流水线，并处理数据冲突与分支跳转；③ 集成 UART、GPIO 等基础外设用于系统控制与结果输出。

· 高阶任务：① 两路组相联的指令与数据 Cache，优化总线突发传输效率；② 动态分支预测（如 BTB）与完整的 RISC-V 异常/中断处理机制；③ 基于 RISC-V 的边缘 AI 加速单元，移植轻量模型并完成识别类应用。

1.2 完成情况对照

任务	要求	完成情况	主要证据
基础①	RV32I 通用处理单元	完成（并扩展 Zicsr、M）	37 条 RV32I 全部实现；t01~t07 覆盖
基础②	≥3 级流水 + 冲突/跳转处理	完成（五级 + 全转发 + BTB）	t02/t07/t12；CoreMark 可运行

任务	要求	完成情况	主要证据
基础③	UART/GPIO 等外设	完成（UART/GPIO/TIMER/测试寄存器）	t09；CoreMark 串口输出
高阶①	2 路组相联 I/D Cache、总线突发	完成	t10/t11；缺失代价 3 拍；BRAM 推断 28 块
高阶②	动态分支预测 + 完整异常/中断	完成（BTB + 2 位计数器；外部中断、向量 mtvec）	t08/t13；IPC 0.59→0.98
高阶③	边缘 AI 加速单元 + 轻量模型	基础版完成（INT8 FC/RELU/ARGMAX + MNIST）	t14；MNIST 19/20

1.3 主要指标

指标	数值	说明
流水线级数	5	IF / ID / EX / MEM / WB
指令集	RV32I + Zicsr + M	37 条基础指令 + 8 条乘除 + CSR
Cache	2 KB × 2 路 × 2 套	16 B/行，伪 LRU，写穿
分支预测	64 项 BTB + 2 位计数器	预测正确时零冲刷
主存	64 KB + 64 KB	4096 × 128 位，BRAM
CoreMark	357 397 ticks / 迭代	CRC 与官方一致，≈2.80 CoreMark/MHz
MNIST	19/20 = 95%	加速器实测，单帧约 6.5k 拍
综合资源	10 919 LUT / 3 726 FF / 28 BRAM / 20 DSP	Artix-7 OOC 估算

2 系统总体架构

2.1 总体结构

系统由 RISC-V 内核、两级存储层次（Cache + 主存）、外设子系统与 AI 加速器组成，如图 1 所示。内核通过独立的取指与访存接口连接 I-Cache 与 D-Cache；Cache 缺失时以整行突发方式访问 128 位宽的 BRAM 主存。外设与加速器挂在同一套数据总线上，由地址译码区分。

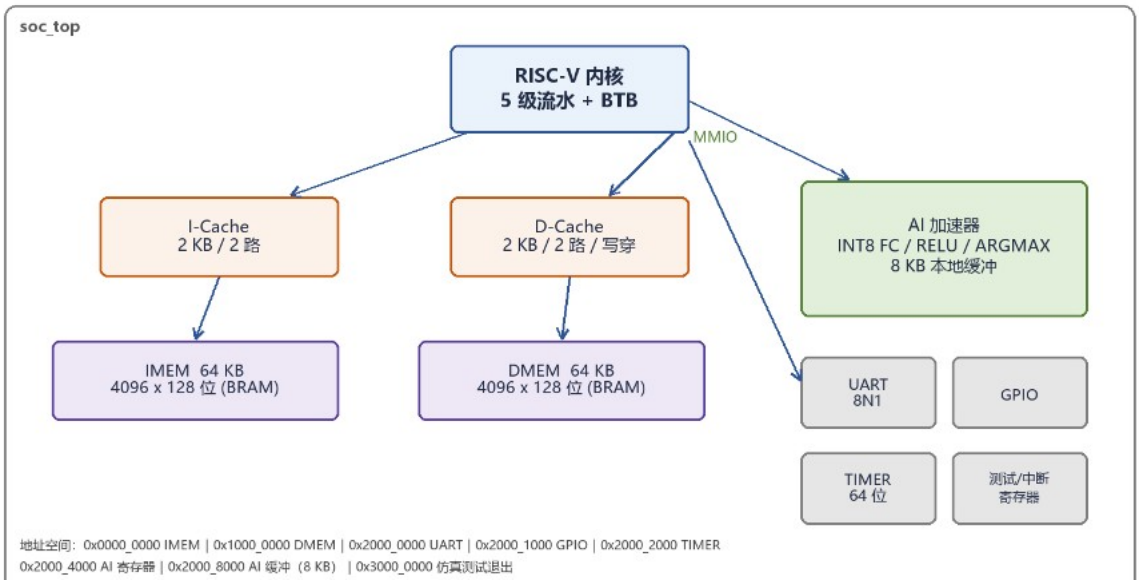


图 1 SoC 总体结构

2.2 地址空间

起始地址	大小	区域	说明
0x0000_0000	64 KB	IMEM	复位向量 0x0，异常向量默认 0x40
0x1000_0000	64 KB	DMEM	数据、栈、堆；字节写使能
0x2000_0000	4 KB	UART	0x00 数据，0x04 状态
0x2000_1000	4 KB	GPIO	0x00 输出，0x04 输入，0x08 方向
0x2000_2000	4 KB	TIMER	mtime / mtimecmp 各 64 位
0x2000_3000	4 KB	中断控制	写 bit0 触发 MEIP（自测用）
0x2000_4000	4 KB	AI 寄存器	CTRL/OP/K/N/WA/XA/YA/CYC/ARG
0x2000_8000	8 KB	AI 缓冲	权重、输入、激活、结果
0x3000_0000	—	测试退出	写 1 为通过，写 code<<1 为失败

2.3 模块划分

层次	模块	职责
内核	riscv_core / decoder / alu / regfile	五级流水、译码、运算、寄存器堆
内核	hazard / btb / divider / mul / csr	冒险控制、分支预测、乘除法、CSR 与异常
存储	icache / dcache / imem / dmem	两路组相联 Cache、128 位主存
外设	uart_tx / uart_rx / gpio / timer	串口、GPIO、64 位定时器
加速	ai_accel	INT8 矩阵-向量、ReLU、ARGMAX
顶层	soc_top / fpga_top	地址译码、板级接口、复位同步

3 处理器内核设计

3.1 五级流水线

内核采用经典五级流水：取指（IF）、译码（ID）、执行（EX）、访存（MEM）、写回（WB）。分支与跳转在 EX 级解析，配合 BTB 预测；异常与中断在 MEM 级统一提交，保证精确异常。结构如图 2 所示。

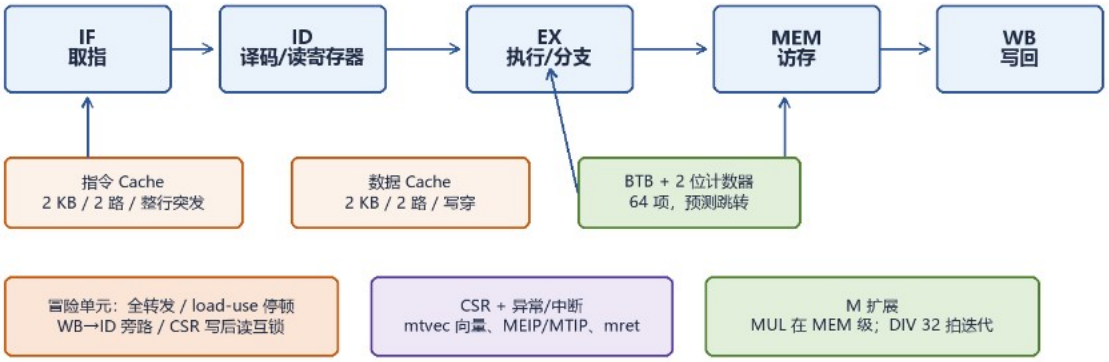


图 2 RV32I 五级流水线结构

3.2 数据冒险处理

机制	说明
EX/MEM 转发	上一条 ALU 结果直接送 EX 的 A/B 口（load 除外）
MEM/WB 转发	写回数据回送 EX，覆盖相隔 2 条的 RAW

机制	说明
WB→ID 旁路	寄存器堆同拍读写时，ID 级直接用写回数据（相隔 3 条的 RAW）
load-use 停顿	EX 为 load 且 ID 依赖其结果时停顿 1 拍，改由 MEM/WB 转发
MUL 互锁	乘法结果在 MEM 末尾产生，依赖指令按 load-use 方式停顿 1 拍
CSR 互锁	csrw 后紧跟 csrr 时让 csrr 在 ID 多停 1 拍

3.3 分支预测

采用 64 项 BTB（index = pc[7:2]，tag = pc[31:8]）加 64 个两位饱和计数器。取指时若表项有效、tag 命中且计数器 ≥2，则下一 PC 取预测目标；EX 级解析后写回真实方向与目标，预测失败时冲刷 IF/ID 与 ID/EX。预测正确时不产生冲刷，循环中的后向分支从第二次迭代起零惩罚。实测 t08 等待循环的 IPC 由 0.59 提升到 0.98，t10 由 0.72 提升到 0.91。

3.4 异常与中断

类型	mcause	触发条件	处理
指令地址非对齐	0	跳转目标 [1:0]≠0	进入 mtvec，记录 mtval
非法指令	2	译码失败	同上
断点	3	EBREAK	同上
Load 非对齐	4	LW/LH 地址未对齐	抑制访存，记录 mtval
Store 非对齐	6	SW/SH 地址未对齐	抑制写使能
机器定时器中断	7（0x8000_0007）	mtime ≥ mtimecmp 且 MIE/MTIE	优先级低于外部中断
机器外部中断	11（0x8000_000B）	irq_ext 且 MIE/MEIE	最高优先级

mtvec 支持直接与向量两种模式：mtvec[1:0]=1 时中断入口为 base + 4×cause，同步异常仍进入 base。陷入时保存 mepc/mcause/mtval，MPIE←MIE、MIE←0、MPP←3；mret 恢复 MIE←MPIE、MPIE←1。WFI 按 RISC-V 规范允许的方式实现为 NOP。

3.5 M 扩展

运算	实现	延迟
MUL / MULH / MULHSU / MULHU	MEM 级用已寄存的操作数计算，DSP 推断	1 拍（依赖再停 1 拍）

运算	实现	延迟
DIV / DIVU / REM / REMU	恢复余数迭代，32 次移位-比较-减法	32 拍
除零 / 溢出	商为 −1、余数为被除数；$-2^{31}/-1$ 得 -2^{31}、余数 0	—

乘法器最初放在 EX 级，与转发 mux、写回 mux 串联，导致关键路径 16 ns、估算频率仅 62 MHz。改为在 MEM 级计算后，EX 路径的逻辑延迟由 10.28 ns 降到 4.08 ns，WNS 由 −6.06 ns 改善到 −2.15 ns，代价是依赖指令多停 1 拍（CoreMark 计时区 +2.7%）。

3.6 指令集覆盖

RV32I 的 37 条指令全部实现，另有 Zicsr（CSRRW/CSRRS/CSRRC/CSRRWI/CSRRSI/CSRRCI）、MRET、FENCE/FENCE.I（按 NOP 处理）与 M 扩展 8 条乘除指令。测试程序对算术、逻辑、移位、比较、分支、跳转、访存、CSR 与异常逐项覆盖，详见第 7 章。

4 存储子系统

4.1 Cache 结构

项目	I-Cache	D-Cache
容量 / 组织	2 KB，64 组 × 2 路 × 4 字	同左
索引 / 标记	set = addr[9:4]，tag = addr[31:10]	同左
替换	优先空路，否则伪 LRU	同左
命中	组合读（LUTRAM），零等待	同左
缺失	128 位整行突发，3 拍	同左
写策略	—	写穿 + 不写分配，行内字节合并

两路组相联消除了直接映射的冲突缺失：用相隔 2 KB 的两个数组交替访问，D-Cache 仅 2 次冷缺失、128 次命中（t11）。容量缺失仍存在：4 KB 顺序读写产生 257 次缺失（t10），符合 2 KB 容量的预期。

4.2 整行突发与 128 位主存

第一版只把 Cache 接口加宽到 128 位、主存仍是 32 位阵列拼接 4 字，综合时因需要 4 个读端口而无法推断 BRAM，LUT 由 6 036 暴涨到 57 495，方案废弃。第二版把主存本身改成 4096 ×

128 位（写仍为 32 位，用 16 路字节使能落到行内），Cache 数据阵列也改为按行存储（128 位/项、单写端口），BRAM 恢复正常。

方案	LUT	FF	BRAM	缺失代价
32 位单字读	6 036	3 120	31.5	5 拍
128 位整行（采用）	7 828	3 159	30	3 拍

实测收益：t01 由 321 拍到 216 拍（−33%），t03 259→175，t06 625→580，t10 16 070→15 260。代价是 LUT 增加约 29%（宽数据阵列与双读端口复制）。

4.3 存储器接口

IMEM/DMEM 均为 BRAM 风格：地址打一拍，读数据下一拍有效；IMEM 只读，DMEM 支持 32 位写与 16 路字节使能。主存由 Cache 以整行突发方式访问，内核不直接连接主存，因此主存可以放心映射为大容量 BRAM。

5 外设与 SoC 集成

外设	寄存器	功能
UART TX	0x2000_0000 写	8N1 发送，写字节触发；状态 bit0=busy
UART RX	0x2000_0000 读	接收字节；状态 bit1=rx_valid
GPIO	0x2000_1000/1004/1008	输出、输入、方向
TIMER	0x2000_2000~200C	mtime 递增，mtimecmp 比较产生 MTIP
中断控制	0x2000_3000	写 1 拉起 MEIP，写 0 清除（自测）
测试退出	0x3000_0000	bit0=1 通过，否则 [31:1] 为失败码

外设与 AI 加速器共用数据总线但绕开 D-Cache：地址译码按 4 KB 页区分，DMEM 段（0x1000_xxxx）才接入 Cache，其余访问直接落到外设，避免 Cache 行填充把外设地址误当作内存。

6 边缘 AI 加速单元

6.1 结构与寄存器

加速器为 memory-mapped 从设备，包含 8 KB 本地缓冲与一个 4 路 INT8 乘累加引擎。CPU 先把权重与输入写入缓冲，再写参数寄存器并启动；引擎完成后置 done，CPU 读回结果。缓冲与引擎分时复用端口，CPU 在 busy 期间不应访问缓冲。

偏移	名称	读写	说明
0x00	CTRL	W/R	W: bit0 启动、bit1 清 done；R: bit0 busy、bit1 done
0x04	OP	W	0=FC，1=RELU，2=ARGMAX
0x08	K	W	FC 的内积长度
0x0C	N	W	RELU/ARGMAX 的元素个数
0x10	WA	W	权重字节偏移（4 字节对齐）
0x14	XA	W	输入字节偏移
0x18	YA	W	输出字节偏移（int32）
0x20	CYC	R	上一次运算的周期数
0x24	ARG	R	ARGMAX 结果

6.2 算子语义

FC：$y[j] = \Sigma_k W[k][j] \cdot x[k]$，一次 4 个输出，W 按 K×4 行优先存放，x 为 K 个 INT8，结果为 4 个 INT32；偏置通过“x 末尾置 1、W 增加一行偏置”折入，不占硬件。RELU：对 YA 起的 N 个 INT32 原地取正。ARGMAX：对 N 个 INT32 求最大值下标（有符号比较）。

6.3 量化流程

在 PC 端用 numpy 训练 196-32-10 的 MLP（输入为 28×28 下采样到 14×14），权重做 per-tensor 对称量化（±127），偏置折算到整数累加域；隐层激活用校准集确定定点缩放系数 M，按 (y·M)>>15 量化为 INT8。该方案把 INT8 推理精度从早期固定右移方案的 65% 提升到 96.19%（10 000 张测试集），已接近浮点模型的 96.26%。

6.4 推理流程与结果

```
CPU: 图像/权重 -> AI 缓冲（sw）
```

```
-> FC(196×4) × 8 组    （第 1 层 196→32）
-> 偏置 + ReLU + 定点缩放（CPU，32 个元素）
-> FC(32×4) × 3 组     （第 2 层 32→10）
-> 偏置 -> ARGMAX（硬件）-> 类别
```

项目	结果
模型	196-32-10 MLP，INT8 权重，权重 6.7 KB
量化精度（Python，10k 测试集）	float 96.26% / INT8 96.19%
RTL 仿真（20 张图）	19/20 = 95%
整程序周期	170 691 拍（含一次权重装载）
单帧周期	约 6.5k 拍（8 组 +3 组 FC 与归约）

7 验证与结果

7.1 验证环境

项目	说明
仿真器 A	Icarus Verilog 12.0，快速回归（sim/run.ps1）
仿真器 B	Vivado Simulator 2023.1，交叉验证（sim/run_xsim.ps1）
汇编器	自研 rvasm.py（RV32I + Zicsr + M，免交叉工具链）
C 工具链	riscv-none-elf-gcc 14.2.0（-march=rv32im_zicsr -O2）
综合	Vivado 2023.1，Artix-7 xc7a35t，OOC 估算（目标器件为 PGL22G）
判据	写 0x3000_0000：bit0=1 通过；否则为失败码

7.2 回归结果（15 项）

测试	覆盖点	周期	退休指令	IPC
t00_smoke	复位、取指、退出通路	19	3	0.16
t01_alu	24 项 ALU/立即数/移位/比较/LUI/AUIPC	216	131	0.61
t02_branch	6 种分支、清空错误路径、前后向循环	258	161	0.62
t03_mem	字节/半字/字、符号扩展、load-use、store 转发	175	99	0.57

测试	覆盖点	周期	退休指令	IPC
t04_jump	JAL/JALR 链接值、嵌套调用、ret	174	81	0.48
t05_csr_trap	CSR 读写、ecall/ebreak/非法指令、mret	171	90	0.53
t06_sort	冒泡排序集成（数据段 + 双层循环）	580	435	0.75
t07_hazard	依赖链、WB→ID 旁路、load-use、CSR 互锁	137	70	0.51
t08_irq	定时器中断、mcause、mret 恢复 MIE	6 499	6 431	0.99
t09_periph	GPIO、mtime/mtimecmp、UART 状态	245	170	0.69
t10_cache	4 KB 顺序读写 + 热点重复访问	15 260	14 622	0.96
t11_conflict	相隔 2 KB 的两数组交替访问	444	405	0.91
t12_muldiv	MUL/DIV/REM 边界（除零、溢出、符号）	532	118	0.22
t13_trap_ext	向量 mtvec、外部中断、非对齐访存、WFI	180	89	0.49
t14_ai	加速器 FC / RELU / ARGMAX	206	119	0.58

15 项全部通过。周期数包含冷缺失与流水线冲刷；IPC 为退休指令数与周期数之比，其中 t08 接近 1.0 是因为其主体为无分支的等待循环。

7.3 CoreMark

项目	结果
版本 / 配置	CoreMark 2K performance run，ITERATIONS=1
seedcrc	0xe9f5（与官方一致）
crclist / crcmatrix / crcstate / crcfinal	0xe714 / 0x1fd7 / 0x8e3a / 0xe714

项目	结果
计时区	357 397 拍
整程序	453 652 拍 / 388 324 条指令（IPC 0.86）
折算性能	≈2.80 CoreMark/MHz（50 MHz 仿真时钟）

EEMBC 要求单次运行不少于 10 秒，仿真相下达不到，因此以官方 CRC 值作为功能正确性的判据，性能按 mcycle 拍数折算。2 次迭代的计时区为 701 416 拍（每次 350 708），与 1 次迭代的结果偏差 0.06%。

7.4 双仿真器交叉验证

t01/t02/t05/t06/t09/t10/t12/t13 等用例在 Vivado xsim 2023.1 下重新编译、精化、仿真，结果与 Icarus Verilog 一致，均为通过。

7.5 综合资源

配置	LUT	FF	BRAM	DSP	10 ns WNS
内核（不含存储）	2 468	1 994	—	—	−0.34 ns
SoC（无 AI）	7 828	3 159	30	12	−3.10 ns
SoC（含 AI）	10 919	3 726	28	20	（未单独跑时序）

以上为 Artix-7 OOC 综合估算，布线延迟占比约 68%，真实频率需在 PDS 完成布局布线后确认。AI 加速器约增加 3.1K LUT（8 KB 缓冲按 LUTRAM 实现约占 2K）与 8 个 DSP。

7.6 典型问题与解决

现象	根因	解决
寄存器读到旧值	WB→ID 距离 3 的 RAW 未旁路	ID 级增加写回旁路
csrw 后 csrr 读旧值	CSR 读在 EX、写在 MEM	增加 CSR 写后读互锁
仿真无输出、模块“消失”	文件带 UTF-8 BOM，iverilog 丢弃后续内容	禁止 BOM
Cache 缺失数≈循环次数	分支重定向中止行填充，反复重填	填充不再因重定向中止
CoreMark 卡在启动	128 位主存字节使能复制到 16 个通道	改为平移 4 位 BE
MNIST 结果恒为 0	8 KB 缓冲跨两个 4 KB 页，译码只匹配一个	按 8 KB 对齐译码
FC 启动后立即返回	done 为粘滞位，读到上一轮状态	启动后延时再轮询 busy/done

现象	根因	解决
INT8 精度仅 65%	激活固定右移导致饱和	按校准集定点缩放

8 代码组织与复现

8.1 目录结构

代码与文档分开放置：报告、图表、仿真日志位于“文档”目录，全部 RTL、测试平台、软件与脚本位于“代码”目录，二者互不混杂。

```
FPGA 创新大赛-紫光同创 RISC_V_CPU/
├─ 文档/
│    ├─ 项目总结报告.docx        本报告
│    ├─ make_figs.py             生成报告插图
│    ├─ make_report.py           生成报告
│    ├─ 图/                      架构图（PNG）
│    └─ 仿真日志/                原始仿真输出（txt）
└─ 代码/
     ├─ rtl/core/               CPU 内核（流水线、译码、 ALU、冒险、BTB、CSR、乘除）
     ├─ rtl/mem/                Cache 与 128 位主存
     ├─ rtl/ai/                 AI 加速器
     ├─ rtl/periph/             UART / GPIO / TIMER
     ├─ rtl/soc/                SoC 顶层与板级顶层
     ├─ tb/                     自检测试平台
     ├─ sw/tests/               15 个汇编测试
     ├─ sw/tools/               汇编器、hex 拆分工具
     ├─ sw/c/                   裸机 C 环境、bench.c、mnist.c、CoreMark
     ├─ sw/ai/                  MNIST 训练/量化脚本与生成的头文件
     ├─ sim/                    回归与 xsim 脚本
     ├─ constraints/            PDS 引脚/时序约束模板
     └─ doc/                    设计与验证文档（Markdown）
```

8.2 主要文件清单

相对路径（代码/）	行数	说明
README.md	188	
doc/01_架构设计.md	259	
doc/02_验证报告.md	212	
doc/03_后续路线.md	159	
rtl/ai/ai_accel.v	269	INT8 矩阵-向量加速器
rtl/core/riscv_alu.v	32	32 位 ALU
rtl/core/riscv_btb.v	62	BTB + 两位饱和计数器
rtl/core/riscv_core.v	666	五级流水顶层与冲突控制
rtl/core/riscv_csr.v	180	CSR、异常/中断入口、计数器
rtl/core/riscv_decoder.v	257	指令译码与控制信号
rtl/core/riscv_defs.vh	93	全局常量（opcode/ALU/CSR/异常码）
rtl/core/riscv_divider.v	116	恢复余数除法器

相对路径（代码/）	行数	说明
rtl/core/riscv_hazard.v	87	转发、停顿、CSR 互锁
rtl/core/riscv_lsu.v	59	访存对齐与符号扩展
rtl/core/riscv_mul.v	34	INT8→INT32 乘法单元（MEM 级）
rtl/core/riscv_regfile.v	35	32×32 寄存器堆
rtl/mem/dcache.v	139	两路组相联数据 Cache
rtl/mem/dmem.v	59	64 KB 数据主存（128 位行，字节使能写）
rtl/mem/icache.v	111	两路组相联指令 Cache
rtl/mem/imem.v	42	64 KB 指令主存（128 位行）
rtl/periph/gpio.v	40	GPIO 输出/输入/方向
rtl/periph/timer.v	49	64 位定时器与中断
rtl/periph/uart_rx.v	78	8N1 串口接收
rtl/periph/uart_tx.v	82	8N1 串口发送
rtl/soc/fpga_top.v	57	板级顶层（复位同步、LED/按键）
rtl/soc/soc_top.v	308	SoC 顶层与地址译码
sw/ai/mnist_data.h	41	
sw/ai/train_mnist.py	228	MLP 训练与量化导出
sw/c/bench.c	105	裸机 C 基准（筛法/矩阵/CRC/除法）
sw/c/build.ps1	47	
sw/c/build_coremark.ps1	47	
sw/c/crt0.S	38	
sw/c/link.ld	42	
sw/c/mnist.c	163	加速器推理演示
sw/c/uart.c	37	
sw/tests/t00_smoke.s	8	
sw/tests/t01_alu.s	171	
sw/tests/t02_branch.s	155	
sw/tests/t03_mem.s	126	
sw/tests/t04_jump.s	105	
sw/tests/t05_csr_trap.s	119	
sw/tests/t06_sort.s	91	
sw/tests/t07_hazard.s	104	
sw/tests/t08_irq.s	85	
sw/tests/t09_periph.s	64	
sw/tests/t10_cache.s	72	

相对路径（代码/）	行数	说明
sw/tests/t11_conflict.s	41	
sw/tests/t12_muldiv.s	159	
sw/tests/t13_trap_ext.s	114	
sw/tests/t14_ai.s	138	
sw/tools/hexsplit.py	103	Intel HEX → imem/dmem
sw/tools/rvasm.py	592	RV32I 汇编器
tb/tb_soc.v	143	自检平台（TRACE/UART/STATS）

8.3 复现命令

```
# 1)    + 编译 + 全量回归（Icarus Verilog）
cd 代码/sim
powershell -ExecutionPolicy Bypass -File .\run.ps1 -Rebuild

# 2) 单个测试 / 波形
powershell -ExecutionPolicy Bypass -File .\run.ps1 -Test t10  -Vcd

# 3) Vivado xsim 交叉验证
powershell -ExecutionPolicy Bypass -File .\run_xsim.ps1 -Test t01_alu -Rebuild

# 4) CoreMark
cd ../sw/c
powershell -ExecutionPolicy Bypass -File .\build_coremark.ps1 -Iterations 1
cd ../../sim/work
vvp sim.vvp +MAX=3000000 +UART

# 5) MNIST（AI 加速器）
cd 代码/sw/ai && python train_mnist.py     # 训练+量化+生成 mnist_data.h
cd ../c
powershell -ExecutionPolicy Bypass -File .\build.ps1 -Src mnist.c -Extra "-I../ai"
cd ../../sim/work && vvp sim.vvp +MAX=2000000 +UART
```

8.4 工具链

工具	版本 / 路径	用途
Icarus Verilog	12.0（C:\iverilog）	快速回归
Vivado	2023.1（F:\vivado\Vivado\2023.1）	xsim 交叉验证、综合评估
riscv-none-elf-gcc	14.2.0（xPack，临时目录）	CoreMark 与裸机 C
Python	3.x + numpy	汇编器、训练与量化

9 已知限制与后续计划

9.1 已知限制

· Cache 为直接映射改两路组相联的固定结构，未实现关键块优先与非阻塞填充，缺失仍冻结流水线 3 拍。

· Cache 数据阵列采用 LUTRAM，容量继续增大时会挤占逻辑资源；8 KB AI 缓冲同样为 LUTRAM。

· DIV/REM 固定 32 拍，未做提前结束；除法密集场景开销较大。

· 分支预测未实现返回地址栈与更长历史，函数返回仍可能预测失败。

· AI 加速器目前只支持全连接/ReLU/ARGMAX，未实现卷积与池化；模型为 MNIST 级 MLP，未移植 YOLO 等视觉模型，也未接入摄像头/显示。

· 所有结果均为仿真数据；板卡尚未到位，PDS 综合、时序收敛与上板验证未进行。

9.2 后续计划

优先级	内容	预期收益
P0	板卡到位后建 PDS 工程：PLL、引脚约束、BRAM 初始化、上板跑 CoreMark	完成上板验证
P0	乘法器改为带流水寄存器的 DSP（2 拍、每段短）	频率目标 100 MHz
P1	Cache 关键块优先 / 两路组相联 + 非阻塞填充	缺失 3 拍 → 2 拍
P1	AI 加速器扩展卷积与池化，改 8×8 阵列	可跑小型 CNN
P2	AI 缓冲改同步读 BRAM，节省约 2K LUT	资源更宽裕
P2	返回地址栈、除法提前结束	函数密集/除法密集场景提速

附录 A 寄存器与地址映射

CSR	地址	说明
mstatus	0x300	MIE/MPIE/MPP
mie	0x304	MEIE/MTIE/MSIE
mtvec	0x305	直接/向量模式

CSR	地址	说明
mscratch	0x340	暂存
mepc	0x341	异常返回地址
mcause	0x342	异常/中断原因
mtval	0x343	异常附加信息
mip	0x344	MEIP/MTIP
mcycle/minstret	0xB00/0xB02	64 位计数器（高字 0xB80/0xB82）

附录 B 仿真日志清单

文件	内容
仿真日志/01_回归测试_iverilog.txt	15 项自检的完整输出与汇总表
仿真日志/02_CoreMark_构建.txt	CoreMark 编译输出
仿真日志/02_CoreMark_运行.txt	CoreMark 串口输出（CRC、ticks）
仿真日志/03_MNIST_构建.txt	MNIST 程序编译输出
仿真日志/03_MNIST_运行.txt	20 张测试图的预测/标签与正确数